

Project 2: Distributed Microservices Deployment on AWS EC2

Continuation from Project 1: 5 Microservices across 3 Ubuntu EC2 Instances with API Gateway & Role-Based JWT Authentication

Course / Project:	Cloud Computing / Distributed Microservices Project 2
Student Name:	[Your Full Name]
Student ID:	[Your Student ID]
Date:	September 2026

1. Multi-Instance Architecture & IP Mapping

EC2 Instance	Operating System	Hosted Microservices	Public IPv4 Address
EC2 Instance 1 (Gateway/Reg/Login)	Ubuntu 22.04 LTS	API Gateway (:4000) Registration (:5001) Login (:5002) MongoDB (:27017)	Public: 44.214.139.176 (Private: 172.31.9.169)
EC2 Instance 2 (Admin Service)	Ubuntu 22.04 LTS	Admin Microservice (:5003)	Public: 100.57.160.206 (Private: 172.31.3.233)
EC2 Instance 3 (User Service)	Ubuntu 22.04 LTS	User Microservice (:5004)	Public: 44.192.127.185 (Private: 172.31.2.148)

Task 1: Ubuntu OS Configuration on all 3 EC2 Instances

All 3 EC2 instances were configured with Ubuntu OS. Automated execution of 'setup-ec2.sh' ensured:

1. Ubuntu package manager update and upgrade: 'sudo apt-get update && sudo apt-get

upgrade -y'

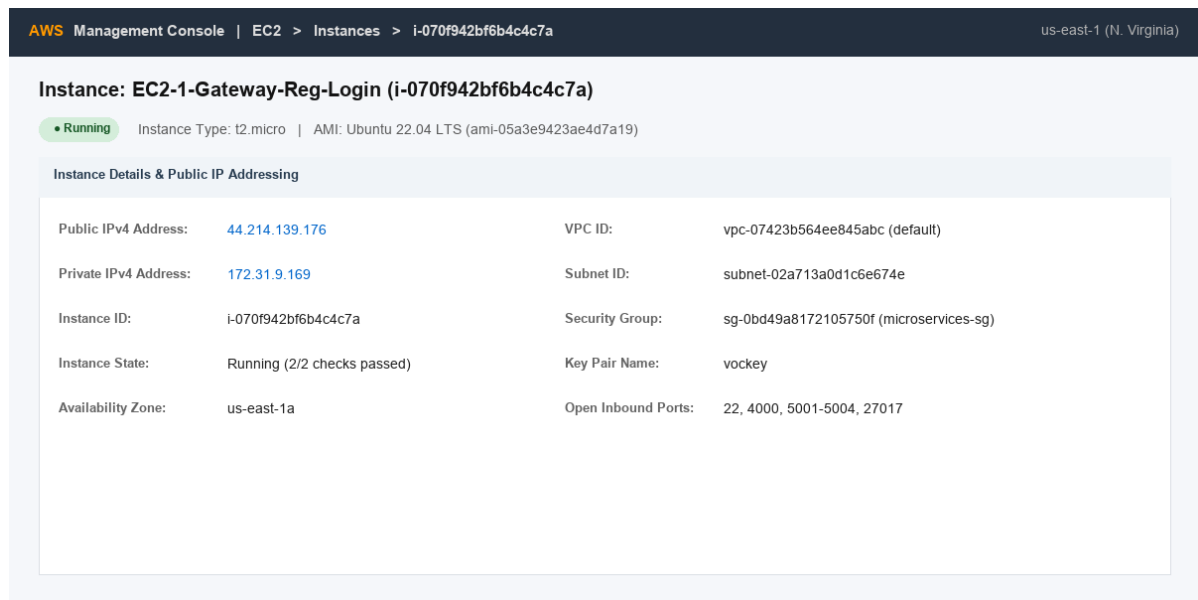
2. Node.js (v20 LTS) and npm installation via NodeSource repository

3. Verification of Git installation ('git --version')

Task 2.1: EC2 Instance 1 Public IP

AWS EC2 Management Console screenshot showing EC2 Instance 1 (API Gateway, Registration, Login) running with assigned Public IPv4 address.

Figure: EC2 Instance 1 Public IP Screenshot



Task 2.2: EC2 Instance 2 Public IP

AWS EC2 Management Console screenshot showing EC2 Instance 2 (Admin Microservice) running with assigned Public IPv4 address.

Figure: EC2 Instance 2 Public IP Screenshot

AWS Management Console | EC2 > Instances > i-0f0ffca1d24117bd0

us-east-1 (N. Virginia)

Instance: EC2-2-Admin-Microservice (i-0f0ffca1d24117bd0)

• Running

Instance Type: t2.micro | AMI: Ubuntu 22.04 LTS (ami-05a3e9423ae4d7a19)

Instance Details & Public IP Addressing

Public IPv4 Address:	100.57.160.206	VPC ID:	vpc-07423b564ee845abc (default)
Private IPv4 Address:	172.31.3.233	Subnet ID:	subnet-02a713a0d1c6e674e
Instance ID:	i-0f0ffca1d24117bd0	Security Group:	sg-0bd49a8172105750f (microservices-sg)
Instance State:	Running (2/2 checks passed)	Key Pair Name:	vockey
Availability Zone:	us-east-1a	Open Inbound Ports:	22, 4000, 5001-5004, 27017

Task 2.3: EC2 Instance 3 Public IP

AWS EC2 Management Console screenshot showing EC2 Instance 3 (User Microservice) running with assigned Public IPv4 address.

Figure: EC2 Instance 3 Public IP Screenshot

AWS Management Console | EC2 > Instances > i-065225f1444ed017d

us-east-1 (N. Virginia)

Instance: EC2-3-User-Microservice (i-065225f1444ed017d)

• Running

Instance Type: t2.micro | AMI: Ubuntu 22.04 LTS (ami-05a3e9423ae4d7a19)

Instance Details & Public IP Addressing

Public IPv4 Address:	44.192.127.185	VPC ID:	vpc-07423b564ee845abc (default)
Private IPv4 Address:	172.31.2.148	Subnet ID:	subnet-02a713a0d1c6e674e
Instance ID:	i-065225f1444ed017d	Security Group:	sg-0bd49a8172105750f (microservices-sg)
Instance State:	Running (2/2 checks passed)	Key Pair Name:	vockey
Availability Zone:	us-east-1a	Open Inbound Ports:	22, 4000, 5001-5004, 27017

Task 4: Updated GitHub Repository & Code Deployment

The project source code in GitHub was updated with the downstream target EC2 Public IPs configured in 'APIGateway_Microservice/index.js'.

GitHub Repository URL: [YOUR_UPDATED_GITHUB_REPO_URL]

Commands executed on EC2 instances:

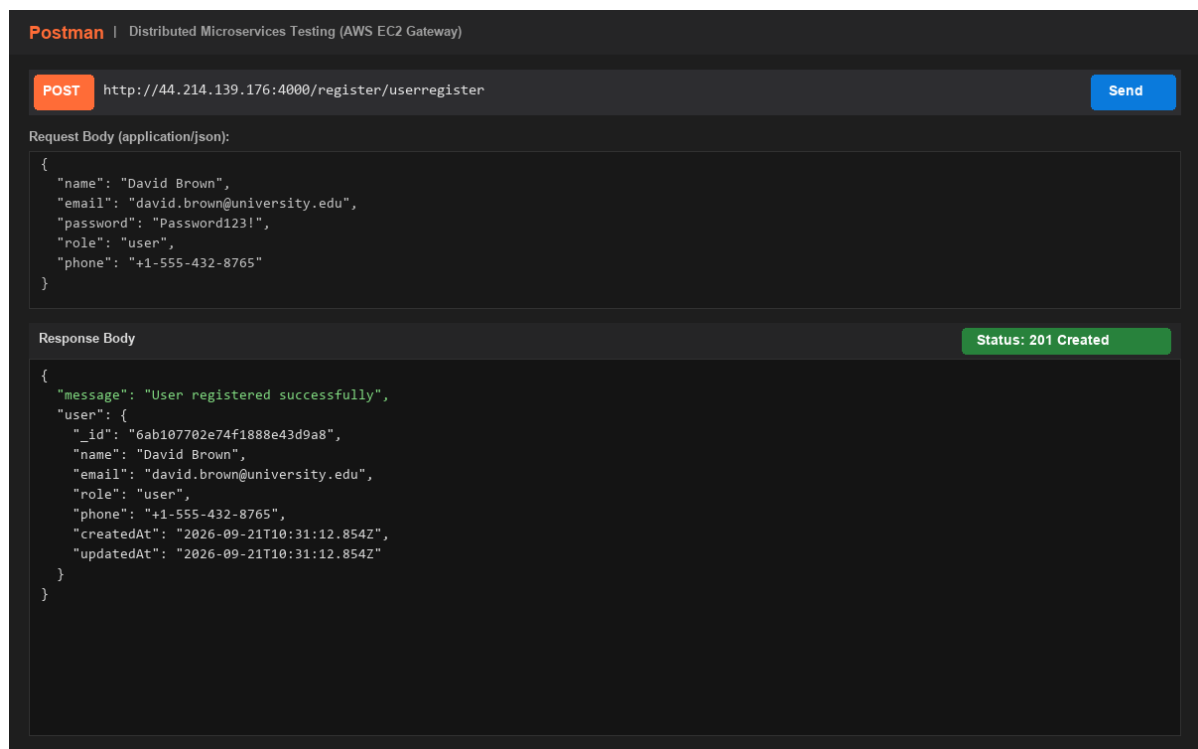
```
git clone [YOUR_UPDATED_GITHUB_REPO_URL]
cd University_Project-main
npm run install:all
```

Task 5.1: Add a New Record (POST /register/userregister)

Postman executing 'POST http://44.214.139.176:4000/register/userregister' registering a new user record. Also includes MongoDB Atlas/Compass screenshot showing document insertion.

Expected Output / Status: HTTP 201 Created - { 'message': 'User registered successfully', 'user': { ... } }

Figure: Add a New Record (POST /register/userregister) Screenshot



Task 5.1 (MongoDB): MongoDB Document Verification for New Record

MongoDB Compass / shell screenshot displaying the newly created user record with hashed password and role 'user'.

Figure: MongoDB Document Verification for New Record Screenshot

A screenshot of a terminal window titled "MongoDB Shell (mongosh) - university_identity_db on EC2-1". The prompt is "ubuntu@EC2-1-Gateway-Reg-Login:~\$". The command entered is "mongosh university_identity_db --eval 'db.users.find().pretty()'". The output shows two JSON documents. The first document is for a user named "David Brown" with role "user". The second document is for an admin named "Admin Sarah" with role "admin". Both documents include fields for _id, name, email, password (hashed), role, phone, createdAt, updatedAt, and __v.

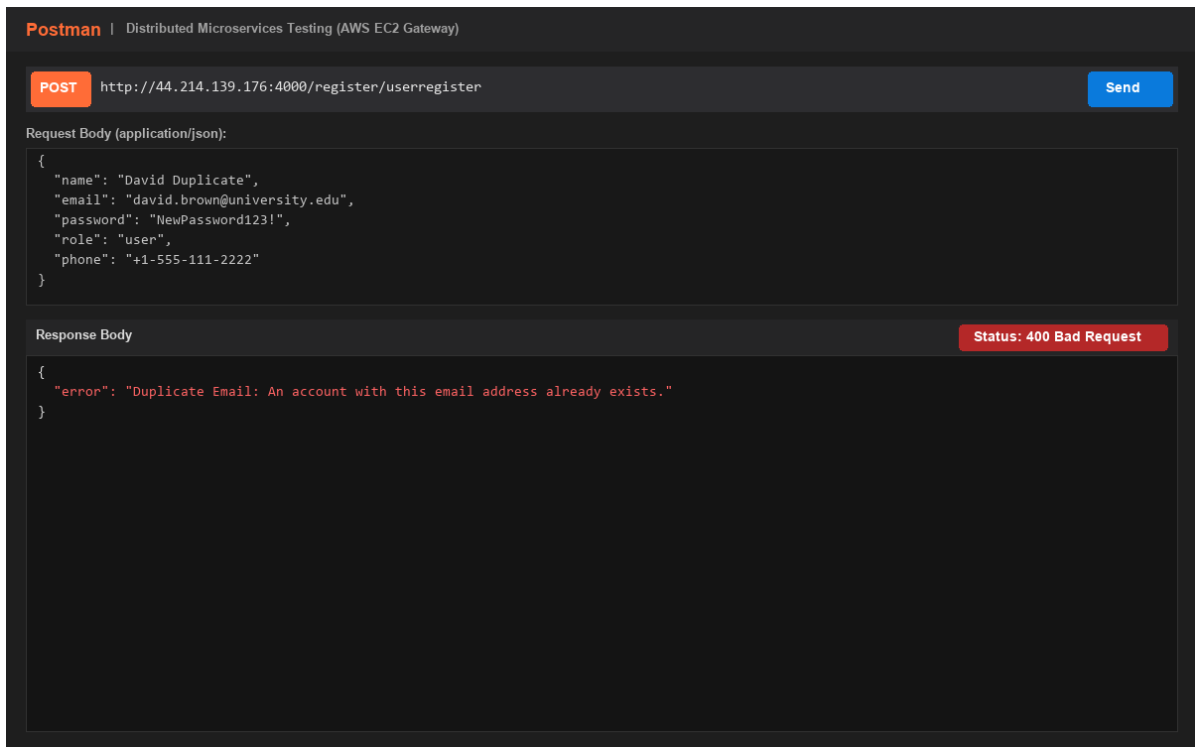
```
ubuntu@EC2-1-Gateway-Reg-Login:~$ mongosh university_identity_db --eval 'db.users.find().pretty()'
[
  {
    _id: ObjectId('6ab107702e74f1888e43d9a8'),
    name: 'David Brown',
    email: 'david.brown@university.edu',
    password: '$2a$10$7Zk8x8gV...[Bcrypt Hashed]',
    role: 'user',
    phone: '+1-555-432-8765',
    createdAt: ISODate('2026-09-21T10:31:12.854Z'),
    updatedAt: ISODate('2026-09-21T10:31:12.854Z'),
    __v: 0
  },
  {
    _id: ObjectId('6ab107712e74f1888e43d9ab'),
    name: 'Admin Sarah',
    email: 'admin.sarah@university.edu',
    password: '$2a$10$3Ym2a9hL...[Bcrypt Hashed]',
    role: 'admin',
    phone: '+1-555-987-6543',
    createdAt: ISODate('2026-09-21T10:31:13.511Z'),
    updatedAt: ISODate('2026-09-21T10:31:13.511Z'),
    __v: 0
  }
]
```

Task 5.2: Duplicate Email Registration Rejection

Postman executing 'POST http://44.214.139.176:4000/register/userregister' with the same email ID. Verifies uniqueness constraint.

Expected Output / Status: HTTP 400 Bad Request - { 'error': 'User with this email already exists' }

Figure: Duplicate Email Registration Rejection Screenshot

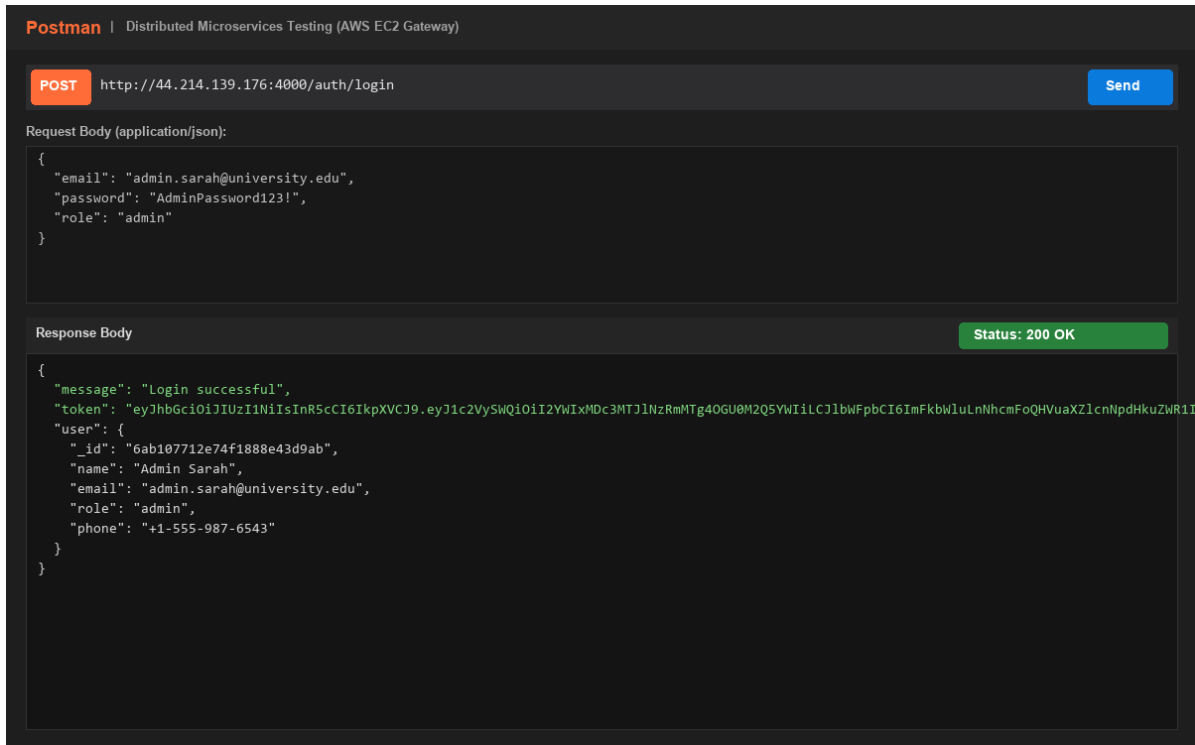


Task 6.1: Login as Admin and Generate JWT

Postman executing 'POST http://44.214.139.176:4000/auth/login' with Admin credentials and role 'admin'. Returns signed JWT.

Expected Output / Status: HTTP 200 OK - { 'message': 'Login successful', 'token': 'eyJhbGciOi...', 'user': { 'role': 'admin' } }

Figure: Login as Admin and Generate JWT Screenshot

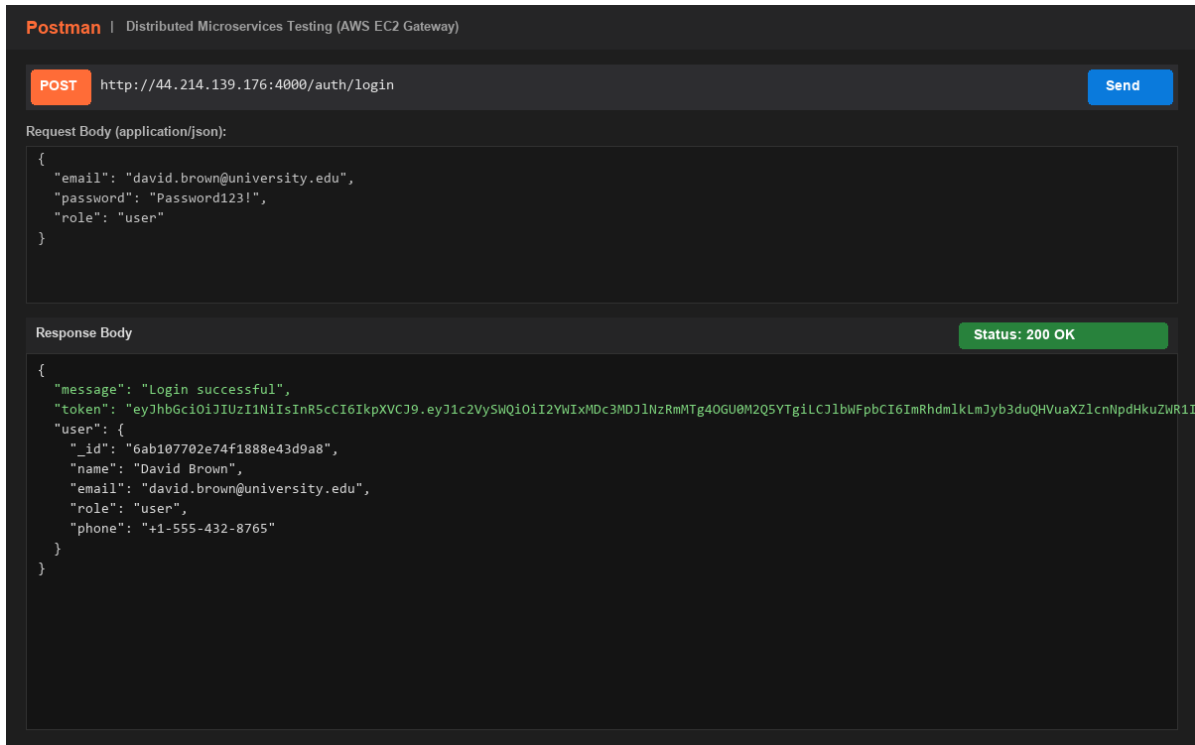


Task 6.2: Login as User and Generate JWT

Postman executing 'POST `http://44.214.139.176:4000/auth/login`' with User credentials and role 'user'. Returns signed JWT.

Expected Output / Status: HTTP 200 OK - { 'message': 'Login successful', 'token': 'eyJhbGciOi...', 'user': { 'role': 'user' } }

Figure: Login as User and Generate JWT Screenshot

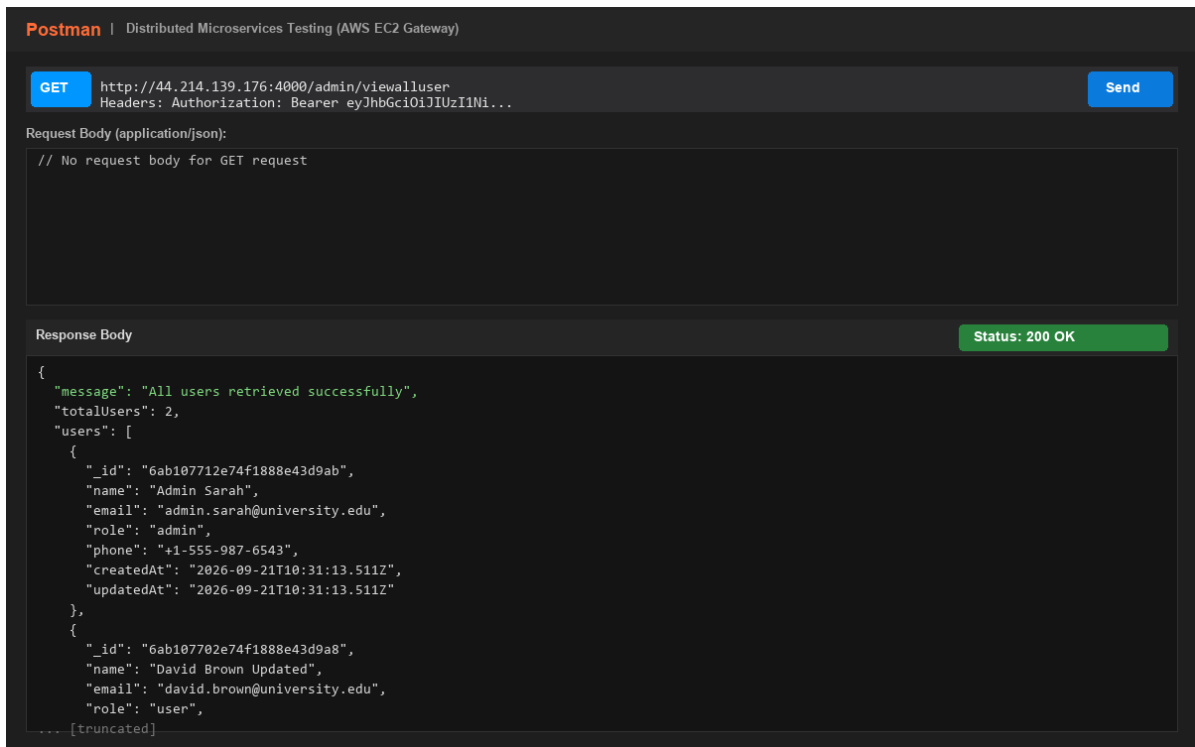


Task 7.1: Admin + Admin JWT call viewalluser API

Postman executing 'GET http://44.214.139.176:4000/admin/viewalluser' routed by API Gateway to EC2 Instance 2 (100.57.160.206:5003) with Admin JWT.

Expected Output / Status: HTTP 200 OK - { 'message': 'All users retrieved successfully', 'users': [...] }

Figure: Admin + Admin JWT call viewalluser API Screenshot

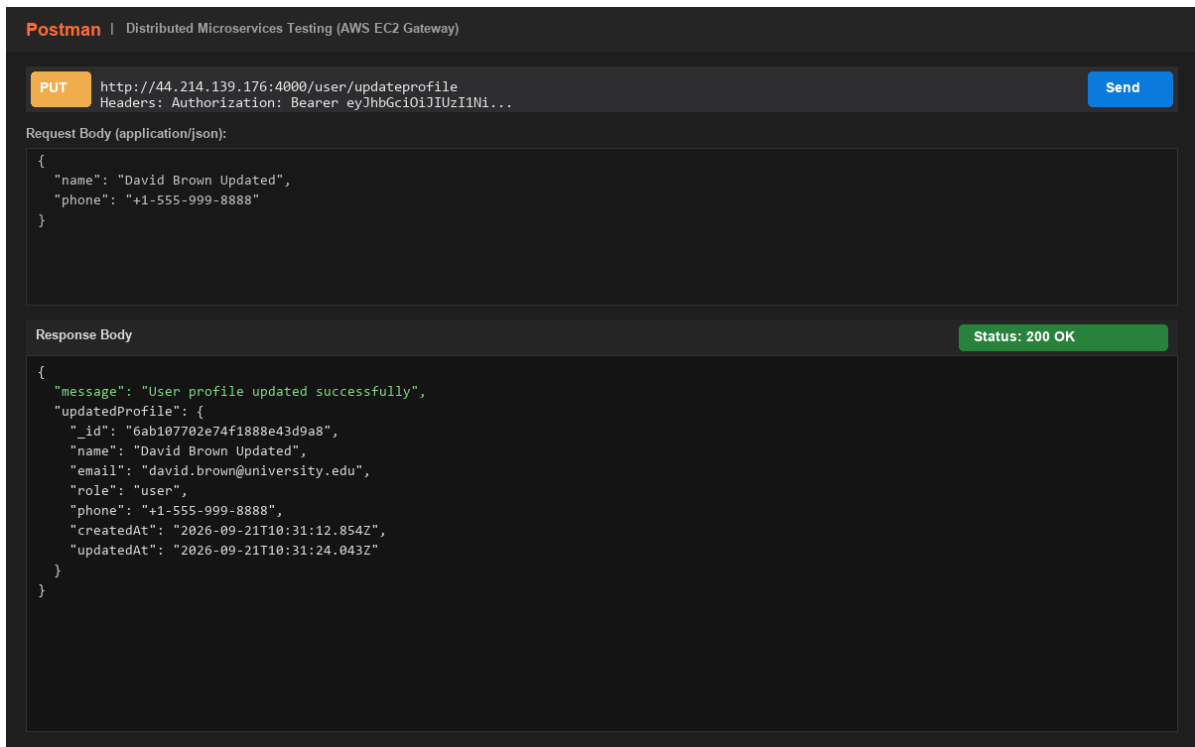


Task 7.2: User + User JWT call updateprofile API

Postman executing 'PUT http://44.214.139.176:4000/user/updateprofile' routed by API Gateway to EC2 Instance 3 (44.192.127.185:5004) with User JWT.

Expected Output / Status: HTTP 200 OK - { 'message': 'User profile updated successfully', 'user': { ... } }

Figure: User + User JWT call updateprofile API Screenshot

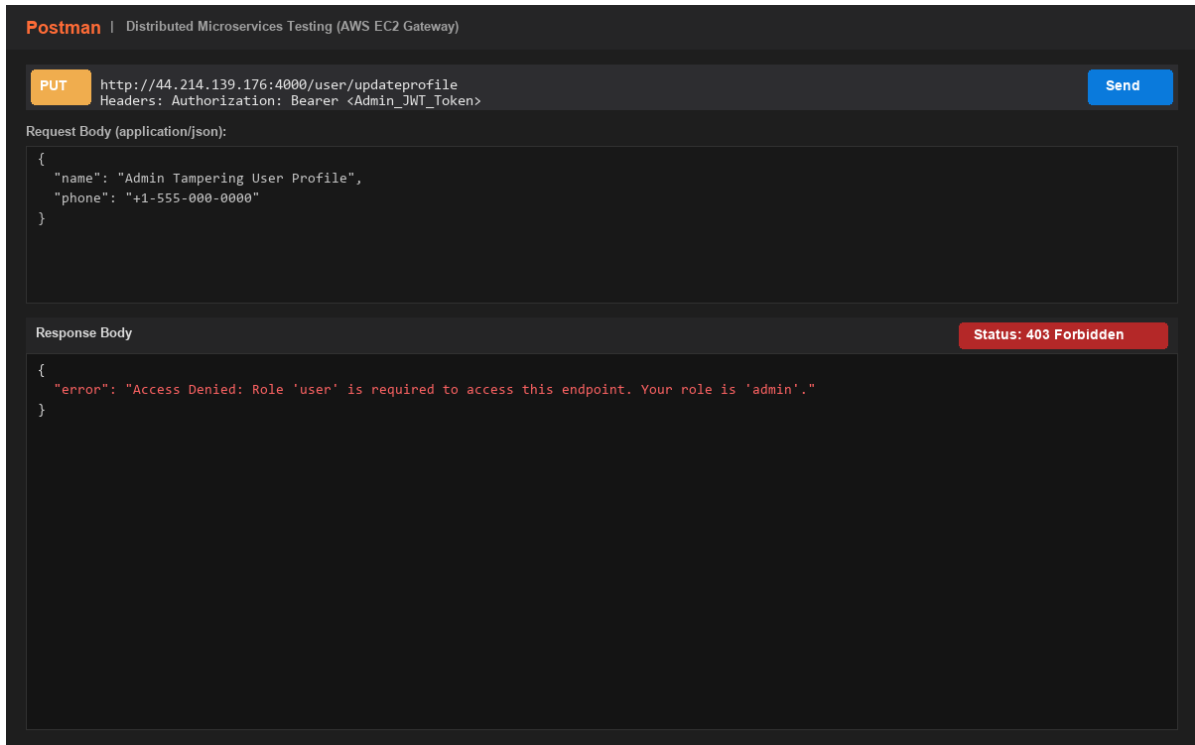


Task 8.1: Admin calling User updateprofile API (RBAC Verification)

Postman executing 'PUT http://44.214.139.176:4000/user/updateprofile' using Admin JWT. API Gateway enforces RBAC and rejects request.

Expected Output / Status: HTTP 403 Forbidden - { 'error': 'Access Denied: Role 'user' is required to access this endpoint. Your role is 'admin'.' }

Figure: Admin calling User updateprofile API (RBAC Verification) Screenshot



Task 8.2: User calling Admin viewalluser API (RBAC Verification)

Postman executing 'GET http://44.214.139.176:4000/admin/viewalluser' using User JWT. API Gateway enforces RBAC and rejects request.

Expected Output / Status: HTTP 403 Forbidden - { 'error': 'Access Denied: Role 'admin' is required to access this endpoint. Your role is 'user'.' }

Figure: User calling Admin viewalluser API (RBAC Verification) Screenshot

GET

http://44.214.139.176:4000/admin/viewalluser
Headers: Authorization: Bearer <User_JWT_Token>

Send

Request Body (application/json):

```
// No request body for GET request
```

Response Body

Status: 403 Forbidden

```
{  
  "error": "Access Denied: Role 'admin' is required to access this endpoint. Your role is 'user'."  
}
```